OWASP ASVS semester audit checklist: map 10 core controls to automated pgTAP, Jest, and GitHub Actions test cases in a compliance matrix

Computer
AppSec debugging runbook: 10 student-app vulnerabilities (SQLi, XSS, broken Supabase RLS) with concrete exploits and fixes
Computer
Show me how to verify the RLS test with pgTAP
Walk through fixing the IDOR vulnerability in the runbook
Show how to wire the security checklist into a pull request template

Created an automated Semester AppSec control package with a 10-control ASVS matrix, pgTAP RLS test template, IDOR remediation guide, and a pull-request security checklist. The key operational point: **RLS must be tested as the application-facing `authenticated` role with realistic JWT claims—not as `postgres` or a service role, which can bypass the policy.** [1] [2]

Files created

File	What it contains
`asvs-5-automated-control-matrix.csv`	10 core controls mapped to ASVS 5.0 chapters, pgTAP/Jest/GitHub Actions, test IDs, acceptance criteria, evidence paths, and owners
`supabase-rls-ac-03-pgtap-template.sql`	AC-03 cross-tenant RLS verification template using positive and negative controls
`IDOR-REMEDIATION-WALKTHROUGH.md`	Vulnerable pattern, authoritative fix sequence, illustrative policy design, and test plan
`PULL_REQUEST_TEMPLATE.md`	Security checklist for `.github/PULL_REQUEST_TEMPLATE.md`
`semester-automated-appsec-controls.zip`	Bundle containing all four files

Ten automated core controls

ID	ASVS 5.0 area	Main tool	What proves success
C01	V8 Authorization	pgTAP	A T1 student cannot read or mutate T2 data
C02	V8 Authorization	pgTAP	No user can self-grant roles, capabilities, or ownership
C03	V8 Authorization	pgTAP + Jest	Foreign-resource RPC/API calls do not leak data or cause effects

ID	ASVS 5.0 area	Main tool	What proves success
C04	V6 Authentication, V9 Tokens	Jest	Expired, forged, wrong issuer/audience tokens are denied
C05	V7 Session Management	Jest + pgTAP	Disabled or revoked subjects lose access within the defined bound
C06	V1 Encoding, V2 Validation	Jest	Invalid inputs cannot alter queries, state, or commands
C07	V3 Web Frontend Security	Jest	User content cannot produce executable DOM/script behavior
C08	V5 Files, V14 Data Protection	pgTAP + Jest	Foreign documents, exports, and storage objects stay inaccessible
C09	V13 Configuration, V15 Architecture	GitHub Actions	No unapproved secrets, critical dependency issues, or unsafe build behavior
C10	V16 Logging/Error Handling	Jest + CI	Security events are recorded without credentials or sensitive raw data

The matrix intentionally starts each control as `NOT_VERIFIED`; actual evidence and test execution must change that status.

Verify AC-03 with pgTAP

AC-03 demonstrates that a synthetic user in tenant T1 cannot access a record in tenant T2. It needs three proof points:

1. **Fixture sanity:** confirm both T1 and T2 records exist outside RLS before testing.
2. **Positive control:** prove the T1 user can access a permitted T1 record.
3. **Negative control:** prove the same T1 user receives zero rows and causes zero mutation on the protected T2 record.

The generated SQL template follows this pattern:

```
begin;
select plan(7);

-- Fixture setup only: privileged local test role.
set local role postgres;

-- Seed and confirm A/T1, C/T2, record_t1, and record_t2.
-- Use your repository's approved fixture helpers.

-- Now test exactly as the application-facing caller.
set local role authenticated;

select set_config(
  'request.jwt.claims',
  '{"sub":"student-a-id","role":"authenticated"}',
  true
);

-- Positive: Student A can see the allowed T1 fixture.
select isnt_empty(
```

```
  $ select 1 from app.student_records where id = 'record-t1' $,
  'T1 student can read authorized T1 fixture'
);

-- Negative: Student A cannot see the T2 fixture.
select is_empty(
  $ select 1 from app.student_records where id = 'record-t2' $,
  'T1 student cannot read T2 record'
);

-- Negative: attempted update causes zero affected rows.
select results_eq(
  $$ with attempted as (
       update app.student_records
       set display_name = 'blocked-test-write'
       where id = 'record-t2'
       returning id
     )
     select count(*) from attempted $$,
  $ values (0::bigint) $,
  'T1 student cannot update T2 record'
);

rollback;
```

An RLS-filtered `SELECT` or `UPDATE` may return **zero rows** instead of raising an exception. That is why the test asserts result emptiness and affected-row count rather than expecting every denial to throw. For rejected inserts, use `throws_ok`; pgTAP/Supabase guidance notes that blocked inserts commonly raise an RLS-policy error, while blocked updates may simply affect no records. [3] [1]

Run the same case through the actual application paths:

- Normal API endpoint.
- Direct Supabase RPC.
- Export/download endpoint.
- Search/list endpoints.
- Storage-object download or signed URL issuance.
- Realtime subscription.
- Cached or derived view.
- Background job or integration path.

Fixing IDOR correctly

The vulnerable pattern is any route that treats an identifier from the client as authorization:

```
// Vulnerable: authentication does not prove access to this record.
const { data } = await supabase
  .from("student_records")
```

```
  .select("*")
  .eq("id", request.params.recordId)
  .single();
```

A logged-in student can replace `recordId` with another user's UUID. UUIDs make discovery harder, but they do **not** prevent IDOR if an identifier is leaked, shared, obtained from an export, or exposed by another endpoint. OWASP recommends per-object, server-side authorization checks for every access attempt. [4] [5]

Correct remediation sequence

1. Derive the authenticated user from a validated server session or JWT.
2. Resolve active tenant membership and capabilities from trusted records.
3. Enforce resource authorization at the database layer with RLS.
4. Use a caller-scoped Supabase client for ordinary user requests so RLS is evaluated.
5. Do not use a service-role client for normal student access. If a privileged backend worker is necessary, independently validate user, tenant, capability, and resource before the bypass-capable query.
6. Repeat authorization enforcement for files, RPCs, exports, realtime, caches, search indexes, views, and background jobs.
7. Add AC-03 pgTAP coverage plus direct API integration tests.

A policy concept should combine verified tenant membership and per-record authority:

```
-- Illustrative only: adapt to confirmed Semester tables and helpers.
create policy "verified tenant member reads own record"
on app.student_records
for select to authenticated
using (
  owner_id = auth.uid()
  and exists (
    select 1
    from app.tenant_memberships m
    where m.tenant_id = student_records.tenant_id
      and m.user_id = auth.uid()
      and m.status = 'active'
      and app.has_capability(
        auth.uid(),
        student_records.tenant_id,
        'student_records.read'
      )
  )
);
```

Staff access should use a separately scoped and tested rule; do not accidentally turn a staff permission into a broad cross-institution read policy. Also review `SECURITY DEFINER` functions, execution grants, function owners, and `search_path` independently: a correct table policy can still be bypassed by a poorly designed privileged function.

Pull request template wiring

Place the generated file at:

```
.github/PULL_REQUEST_TEMPLATE.md
```

GitHub automatically shows a pull-request template when it is stored in `.github`, the repository root, or `docs`; multiple templates can live in `.github/PULL_REQUEST_TEMPLATE/` and be selected with a query parameter.[6]

The template requires authors to declare:

- Whether authorization, RLS, APIs, roles, data flows, uploads, exports, secrets, integrations, CI/CD, or AI authority changed.
- The affected assets, data classes, tenants, and rollback plan.
- Results for type checks, unit tests, secret scanning, SAST, dependency review, and database tests.
- Direct API/RPC, positive authorization, cross-user, and cross-tenant testing.
- Review of RLS, grants, protected columns, `WITH CHECK`, direct function execution, `SECURITY DEFINER`, storage, exports, realtime, and background jobs.
- Relevant ASVS 5.0 requirement IDs, control IDs, evidence links, exceptions, and expiry dates.

A PR template prompts the right disclosures but does not enforce them. Pair it with required branch protection checks for the security workflow, database policy test job, code review, and protected production deployment approval.

⁂

1. https://usebasejump.com/blog/testing-on-supabase-with-pgtap
2. https://unitautogen.com/blog/testing-rls-with-jwt-custom-claims-locally.html
3. https://makerkit.dev/docs/next-supabase-turbo/development/database-tests
4. https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
5. https://community.owasp.org/attacks/insecure_direct_object_reference
6. https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/creating-a-pull-request-template-for-your-repository
7. https://github.com/axolo-co/pull_request_template
8. https://graphite.com/guides/comprehensive-checklist-github-pr-template
9. https://github.com/orgs/supabase/discussions/9311
10. https://stackoverflow.com/questions/52139192/github-pull-requests-template-not-showing
11. https://supabase.com/docs/guides/database/postgres/row-level-security
12. https://dev.to/whoffagents/supabase-row-level-security-in-production-patterns-that-actually-work-2l78
13. https://docs.getdbt.com/blog/analytics-pull-request-template

14. https://axolo.co/blog/p/part-3-github-pull-request-template

15. https://www.harness.io/blog/crafting-a-pull-request-template